

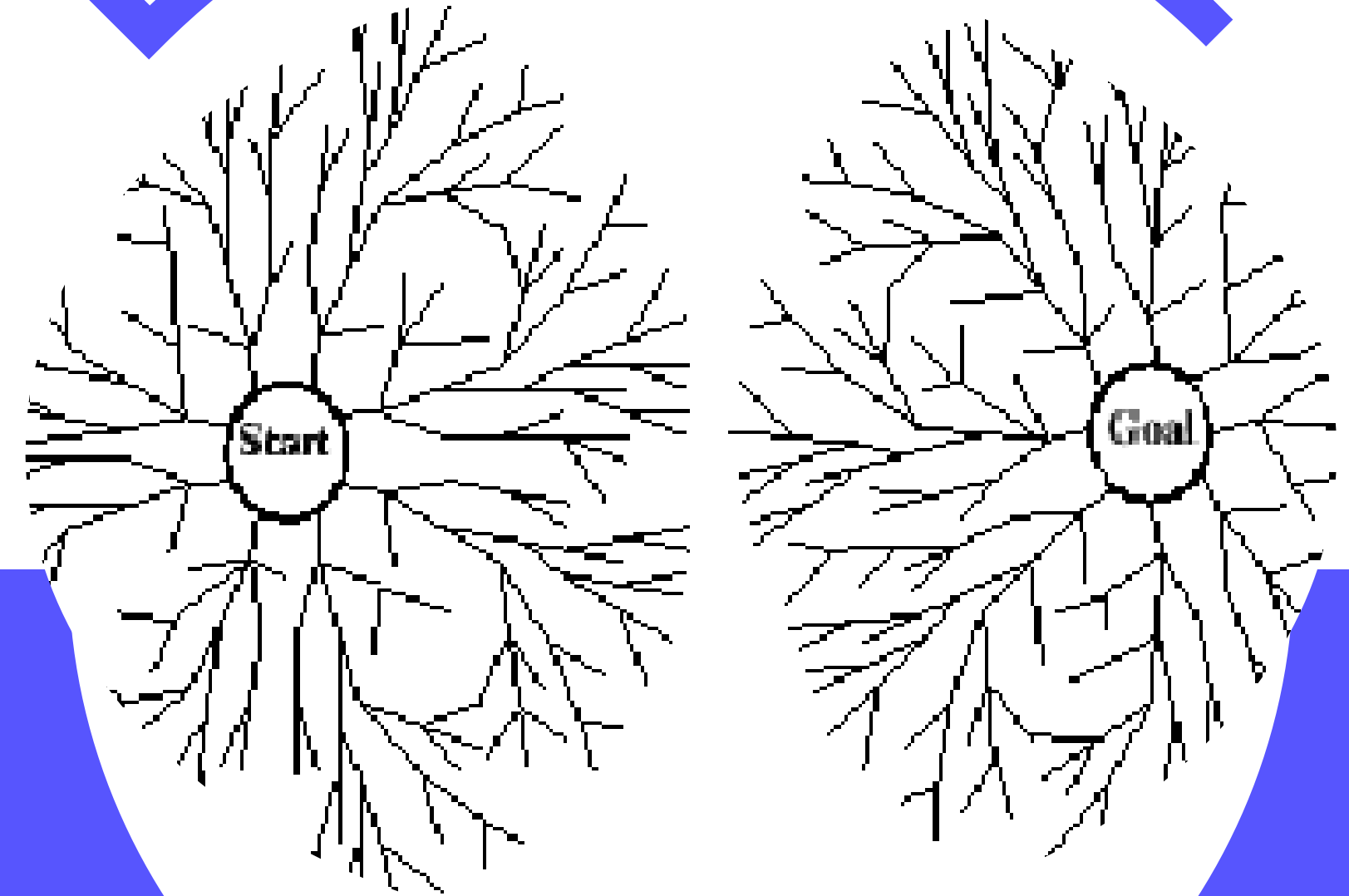
BIDIRECTIONAL SEARCH

Presented by:

Ahmed Ragab Abdelrhman Waheed

Shams Hisham Abdelrhman Mamdouh

Aya Mohamed



1

What is Bidirectional Search

2

Types of Bidirectional Search

3

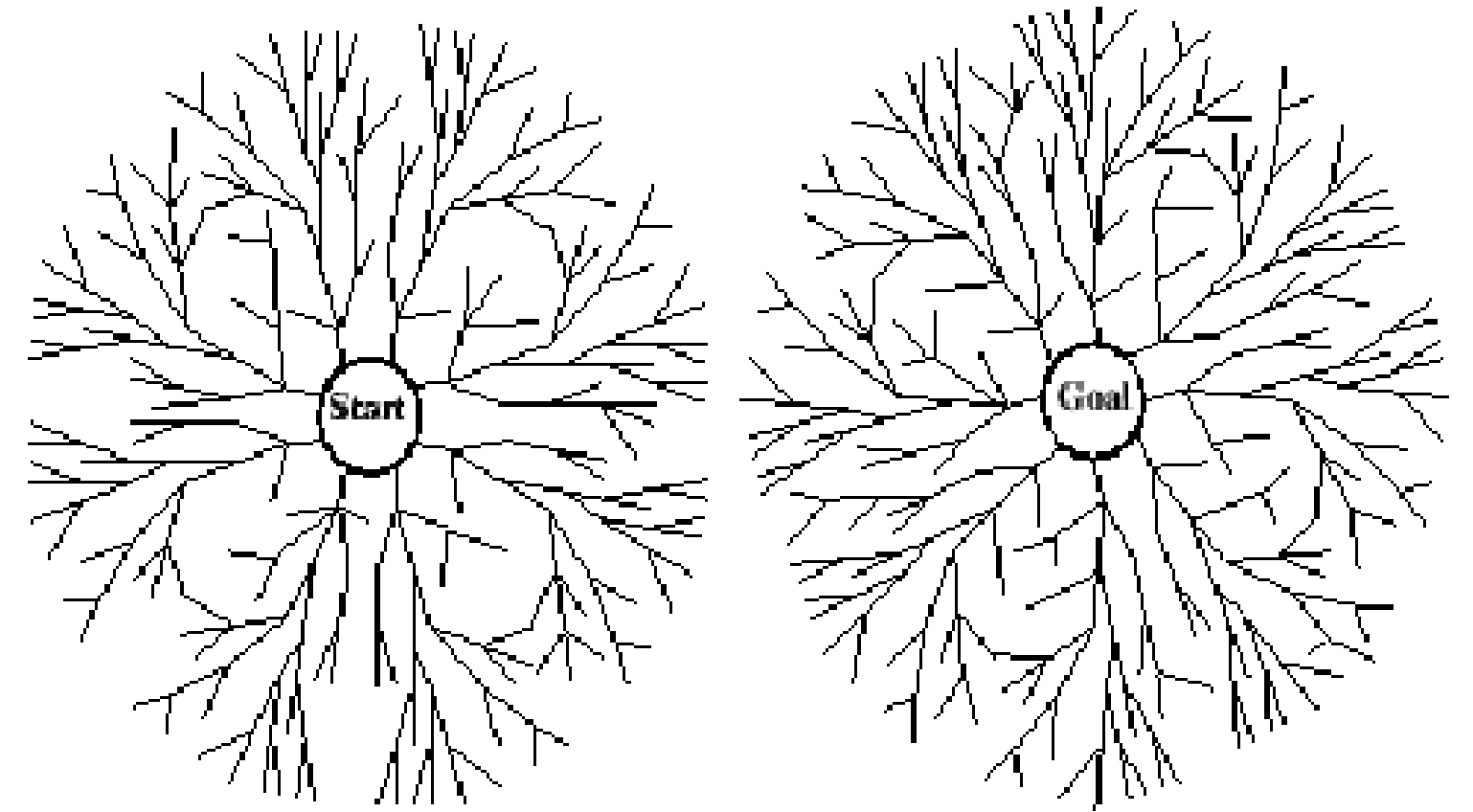
Uninformed Bidirectional

4

Informed Bidirectional

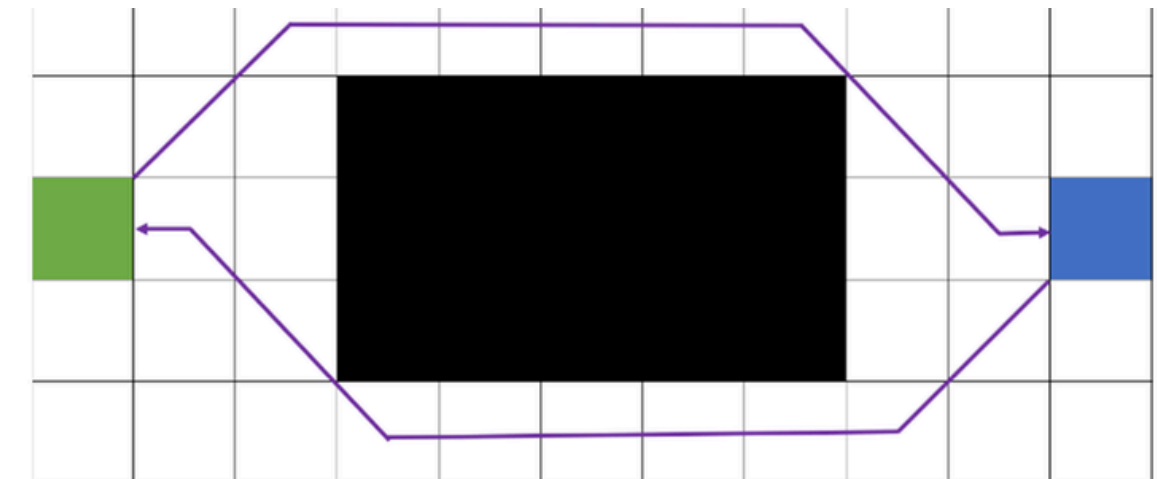
5

Example Problem



2

- ## Example



Types of Bidirectional Search

3

Uninformed

Without heuristic functions, nodes are expanded using the basic approaches

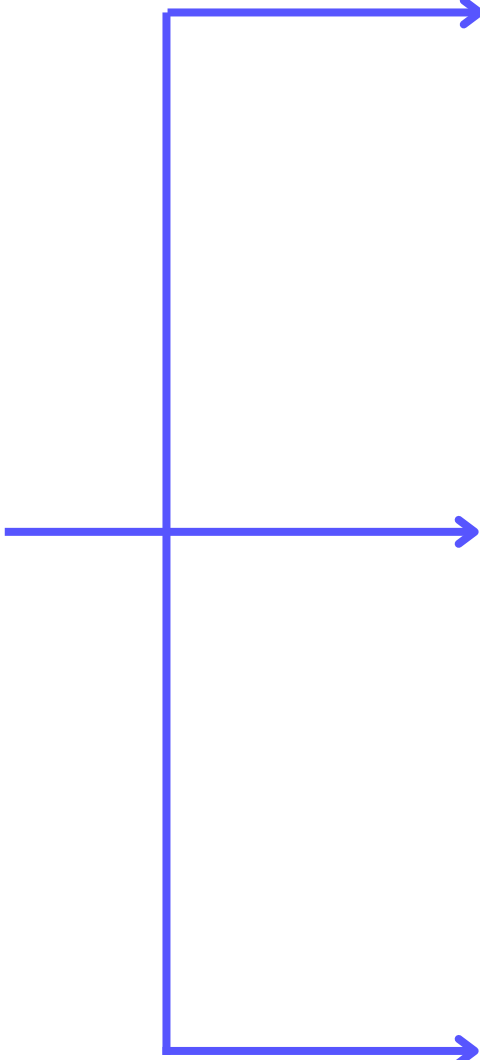
- **Breadth-first-search BFS**
- **Depth-first-search DFS**
- **Uniform-cost-search UCS**

Informed

Enhancing efficiency using heuristic functions through three distinct approaches

- **Front-to-front**
- **Front-to-back**
- **Perimeter search**

Characteristiques



```
graph LR; A[Characteristiques] --> B[Implementing the basic search algorithms, specially BFS, from both ends start and goal states.]; A --> C[For each search, a frontier is explored at each step. The process lasts until an intersection is found.]; A --> D[The algorithm optimizes performance by minimizing the state space and reducing search time];
```

Implementing the basic search algorithms, specially BFS, from both ends start and goal states.

For each search, a frontier is explored at each step. The process lasts until an intersection is found.

The algorithm optimizes performance by minimizing the state space and reducing search time

Evaluation



Completeness

When implemented correctly, it guarantees reaching the goal, as demonstrated by algorithms like BFS and UCS



Optimality

It finds the shortest path if edge costs are equal using (BFS) or handled correctly using (UCS)



Time Complexity

Each search (forward and backward) is exploring only half of the state space reducing the time complexity to $[O(b^{(d/2)})]$



Space Complexity

Since there are two frontiers, for the forward and backward searches, space complexity will also be $[O(b^{(d/2)})]$

Uninformed Bidirectional (3)

6

Advantages compared to informed

Used for simple and general purpose solutions with guaranteed correctness.

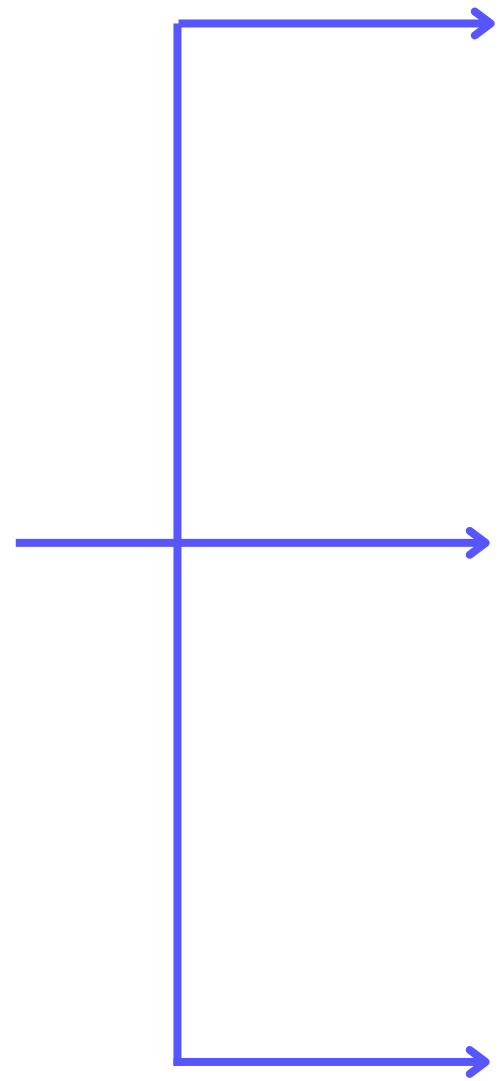
simple domain with uniform costs and with no heuristic function is needed

Disadvantages compared to informed

Explores all paths
wasting time on
irrelevant branches.

Informed approaches are
best suited in large and
structured problems.

Characteristiques



```
graph LR; A[Characteristiques] --> B[Searching from both ends implementing a heuristic function to find the shortest path.]; A --> C[At each step, both the forward and the backward searches expand the node with the min h(n)]; A --> D[Evaluating h(n) is based on three distinct approaches front-to-back, front-to-front, and perimeter.];
```

Searching from both ends implementing a heuristic function to find the shortest path.

At each step, both the forward and the backward searches expand the node with the min $h(n)$

Evaluating $h(n)$ is based on three distinct approaches front-to-back, front-to-front, and perimeter.

Front-to-back approach

- The most intuitive and straightforward approach.
- In the forward search, $h(n)$ estimates the cost from the current state to the goal state.
- In the backward search, $h(n)$ estimates the cost from the current state to the start state.

• Forward estimation $f(n) = g(n) + h(n, \text{goal})$

• Backward estimation $f(n) = g(n) + h(n, \text{start})$

Advantages Front-to-back approach

Is straight-forward and simple to implement than other approaches.

Works well if heuristics of both start and goal states are defined accurately.

Disadvantages Front-to-back approach

Each search does not adapt to whatever the other is doing

May waste time exploring unnecessary paths.

Front-to-front approach

- More intelligent approach.
- Both searches try to meet each other.
- For ex, if the forward search is exploring the area A1, the backward will target that area.

- Forward estimation

$$h(n) = \min(\text{estimation}(n, m)) \quad \text{where } m \text{ in backward frontier}$$

- Backward estimation

$$h(n) = \min(\text{estimation}(n, m)) \quad \text{where } m \text{ in forward frontier}$$

Advantages Front-to-front approach

Improves focus and
reduces wasted
exploration.

Powerful in large state
spaces like puzzles.

Disadvantages Front-to-front approach

More complex to
implement

Need to constantly
update and compare
both frontiers

Perimeter approach

- A little different approach.
- One search usually proceeds till finding a perimeter is found.
- not a common one.

In the perimeter approach, one of the searches expands until it reaches a predefined boundary or region—known as the perimeter. Then, the other search proceeds toward that region, using heuristics ($h(n)$) calculated based on the estimated cost to reach this perimeter area.

Advantages Perimeter approach

Useful if the goal area
is more complicated or
costly to search

The forward perimeter
avoids deep
commitment before the
backward search
confirms a valid path

Disadvantages Perimeter approach

Needs good choice of
perimeter depth

Not widely used or
standardized like the
others

Use Cases of each approach

Front-to-back

Problems with well-defined endpoints

- GPS Navigation
- Robot path-planning

Front-to-front

Puzzles and combinatorial problems with symmetrical properties

- Puzzles

Perimeter

Problems where middle ground or hubs are natural meeting areas

- Large-scale map routing

Advantages compared to uninformed

Can reach intersection point faster and with less space.

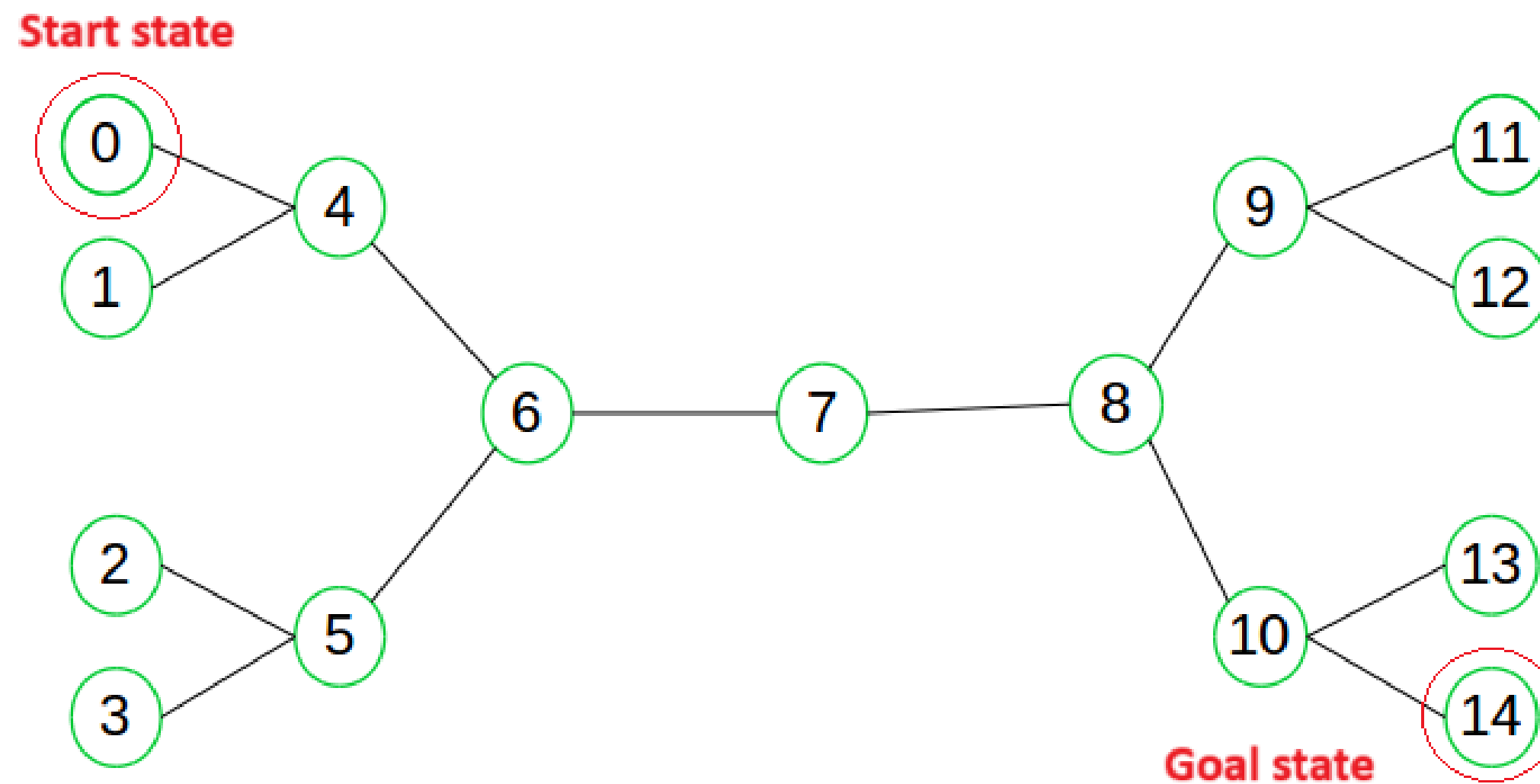
Informed can outperform in cost-based or large-state-space problems.

Considering complexity,
Often achieves less than $O(b^{(d/2)})$ depending on the heuristic quality

Example Problem (1)

20

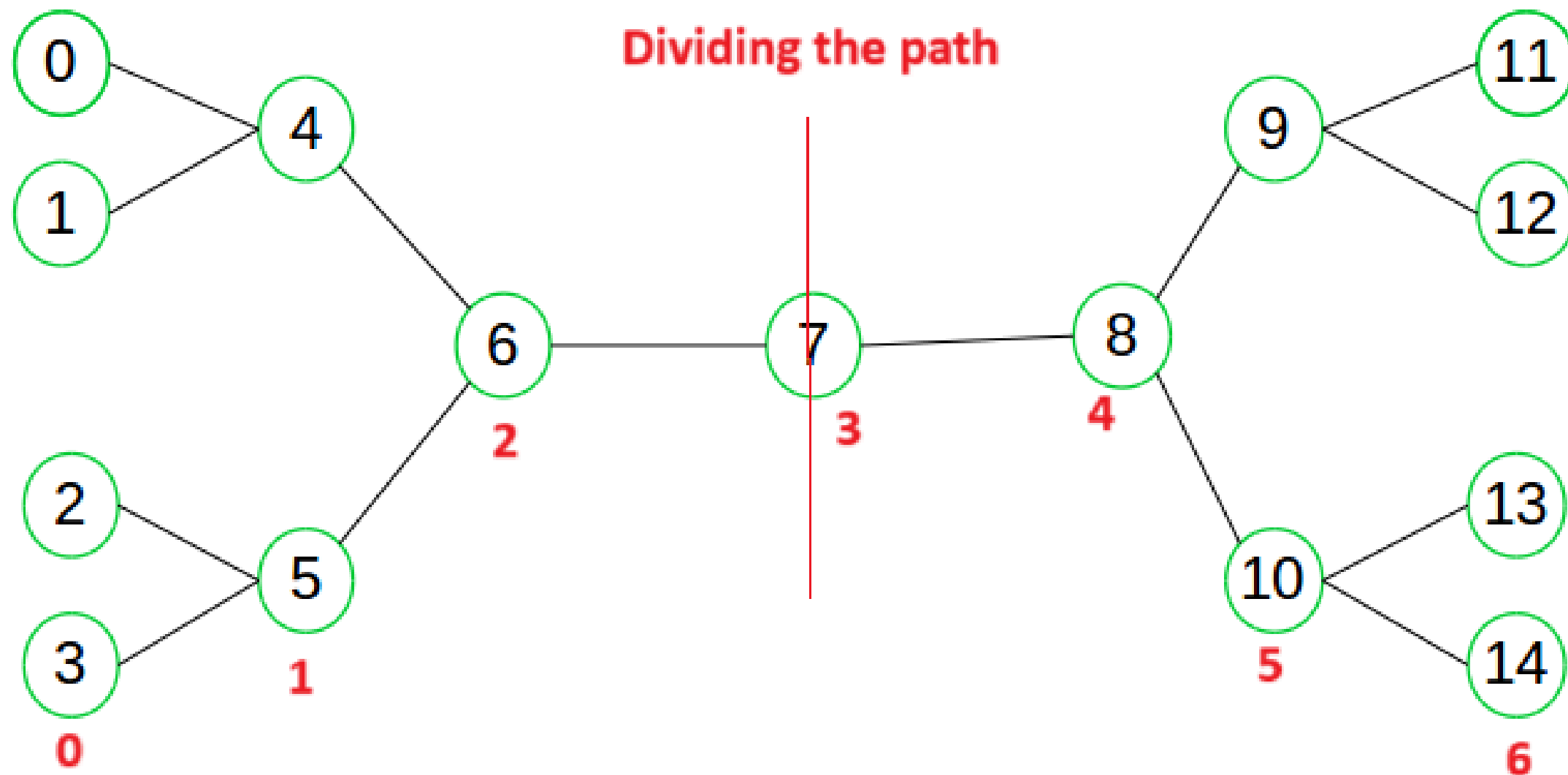
- We will examine an example problem using the BFS algorithm.
- Start state $\rightarrow 0$
- Goal State $\rightarrow 14$



Example Problem (2)

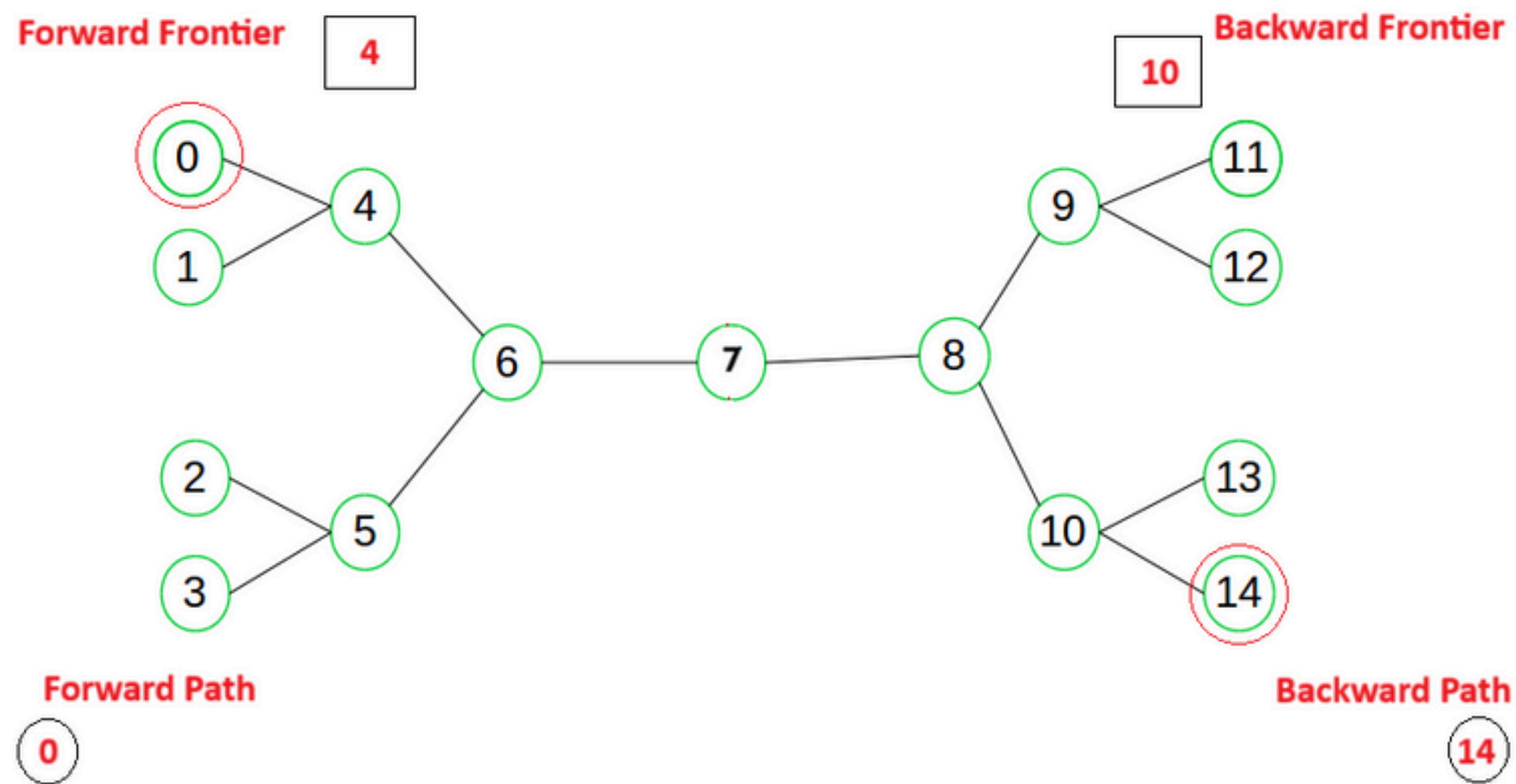
21

- Having 7 levels from 0 to 6
- We divide the path into two equivalent paths
- First Path from 0 to 3.
- Second Path from 3 to 6



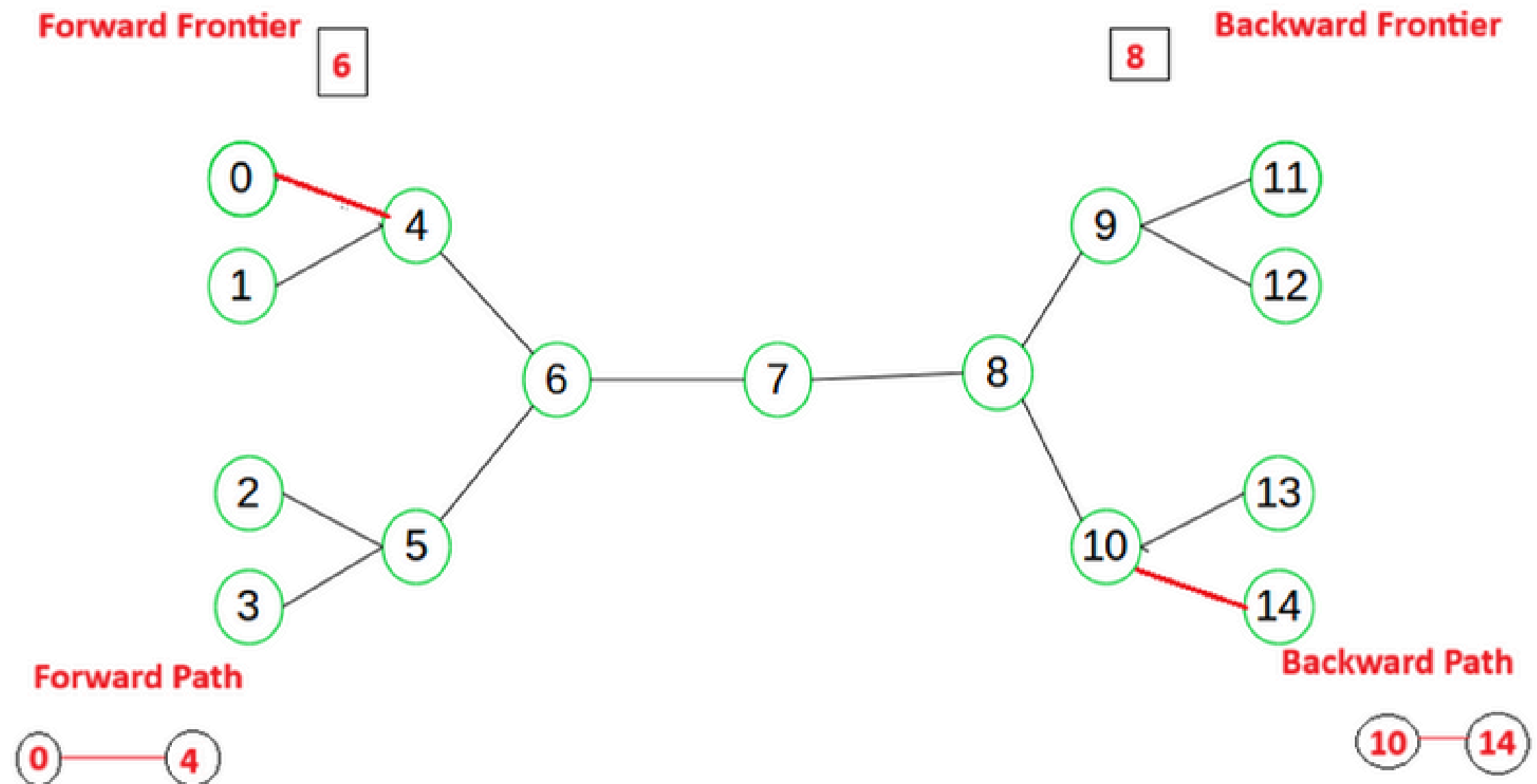
Example Problem (3)

22



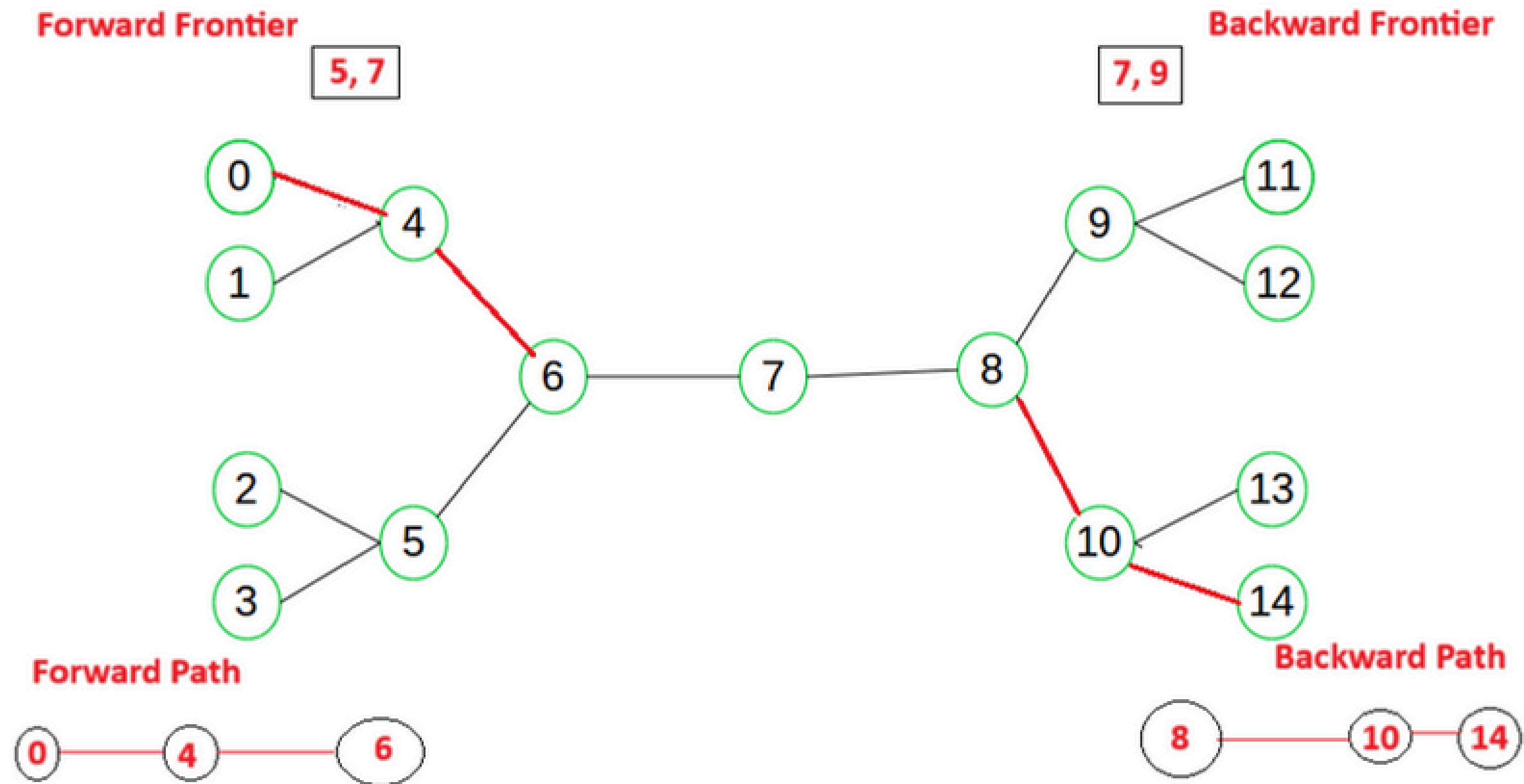
Example Problem (4)

23



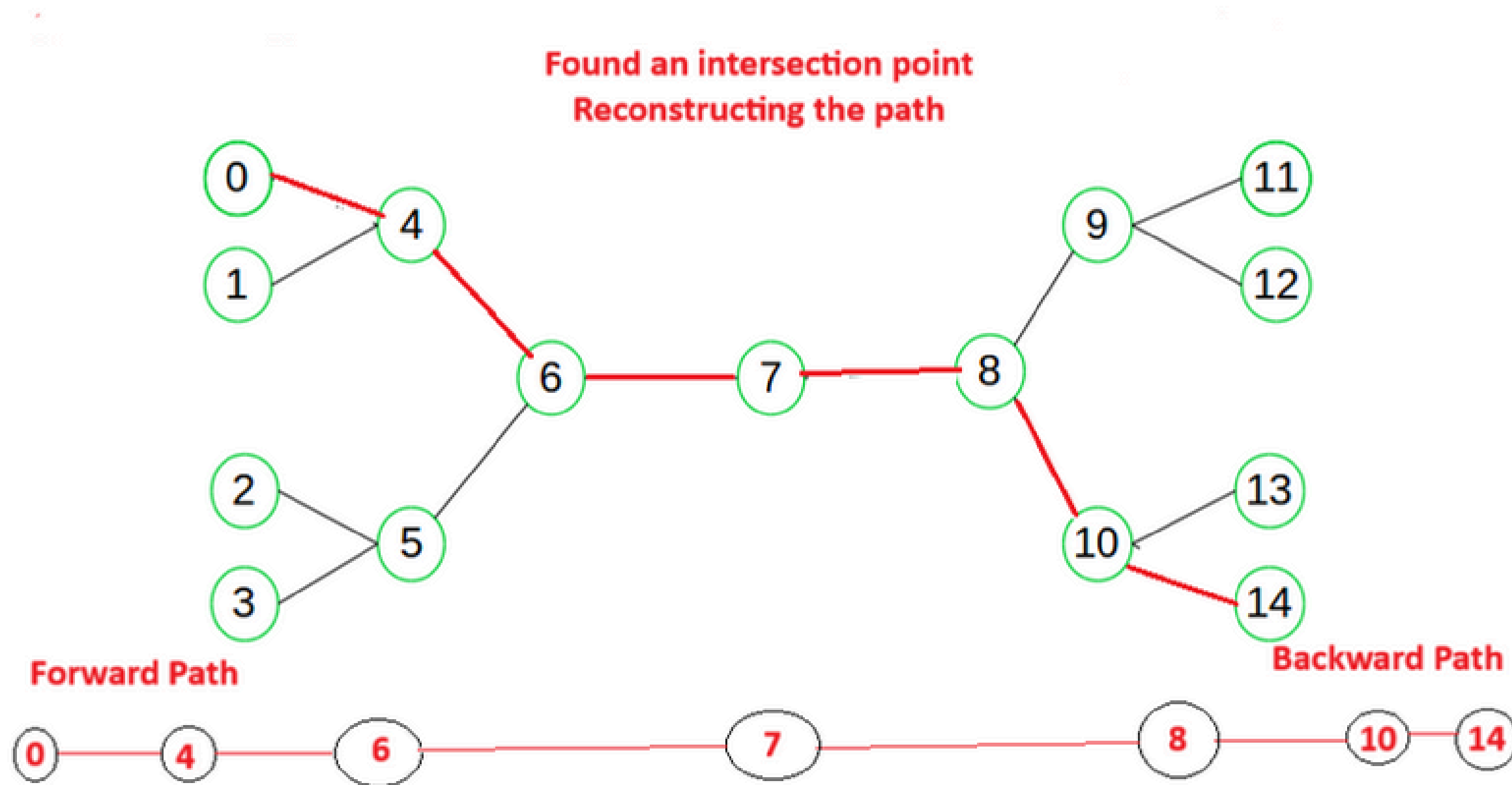
Example Problem (5)

24



Example Problem (6)

25



Bidirectional Search Function:

26

This function performs bidirectional search on the given graph.

- It initializes two queues and visited dictionaries for both the start and goal nodes.
- The search proceeds from both ends simultaneously, checking for overlapping nodes between the two searches.
- When an overlap occurs, it triggers the reconstruction of the path from both directions.

```
def bidirectional_search(graph, start, goal):
    if start == goal:
        return [start]
    visited_from_start = {start: None}
    visited_from_goal = {goal: None}

    queue_from_start = deque([start])
    queue_from_goal = deque([goal])

    while queue_from_start and queue_from_goal:
        node_f_start = queue_from_start.popleft()
        for n in graph[node_f_start]:
            if n not in visited_from_start:
                visited_from_start[n] = node_f_start
                queue_from_start.append(n)
            if n in visited_from_goal:
                return reconstruct_path(visited_from_start, visited_from_goal, start, goal, n)
        node_f_goal = queue_from_goal.popleft()
        for n in graph[node_f_goal]:
            if n not in visited_from_goal:
                visited_from_goal[n] = node_f_goal
                queue_from_goal.append(n)
            if n in visited_from_start:
                return reconstruct_path(visited_from_start, visited_from_goal, start, goal, n)
    return None
```

Reconstruct Path Function:

27

This function reconstructs the shortest path once the two searches meet.

- It traces the path from the meeting node back to the start node and from the meeting node to the goal node.
- The paths are then merged to create the final shortest path.

```
def reconstruct_path(visited_from_start, visited_from_goal, start, goal, n):
    node = n
    path = []
    while node is not None:
        path.append(node)
        node = visited_from_start[node]
    path.reverse()
    path_goal = []
    node = visited_from_goal[n]
    while node is not None:
        path_goal.append(node)
        node = visited_from_goal[node]
    return path + path_goal[1:]

start_node = 0
goal_node = 14
path = bidirectional_search(graph, start_node, goal_node)
print("The Shortest Path Between ", start_node, " & ", goal_node, "IS:", path)
```

[Wikipedia - Bidirectional Search](#)

[GeeksForGeeks - Bidirectional Search](#)



ELE335 (Artificial Intelligence)

TEAM MEMBERS

Presented by:

Ahmed Ragab Abdelrhman Waheed

Shams Hisham Abdelrhman Mamdouh

Aya Mohamed

Directed by Dr. Mohamed Rehan

